Fatima Jinnah Women University

Department of Computer Sciences

Lab 05: Build, Test, and Package Applications with IntelliJ IDEA and Docker

Course	DevOps (CS-649 Cloud DevOps)
Instructor	Sir Waqas Saleem
Student Name	Kiran Siddique
Registration No.	2023-BCS-052
Section	A
GitHub Username	KiranSiddique
Submission Repository	DevOps (Labs/Lab05)
Date of Submission	10 November 2026

Lab Overview

Item	Detail
Lab	Lab 05: Build, Test, and Package Applications with IntelliJ IDEA and Docker
IntelliJ setup repository	Intellij-Idea-docker (IntelliJ IDEA 2026.2.3 in the browser desktop)
Submission directory	DevOps / Labs/Lab05
VM	Ubuntu Server 26.04.1 LTS, user kiransiddique, hostname ubuntu
Docker Engine	29.1.3
Docker Compose	v5.7.0
Git	2.53.0
Maven (system)	3.9.12, running on Java 21.0.12.1
Gradle	7.0 (Gradle Wrapper), Gradle JVM set to JDK 16.0.2

Task List

Task 1: Clone and Launch IntelliJ IDEA

Task 2: Build and Run the Maven Project

Task 3: Build and Run the Gradle Project

Task 4: Install and Run the Node.js Project

Task 5: Build the React and Node.js API Artifact

Task 6: Build a Docker Image for the Node.js Project

Exam Evaluation Questions

Summary

Task 1: Clone and Launch IntelliJ IDEA

I cloned the supplied IntelliJ Docker repository on the Ubuntu server, built the desktop container with the setup script, and opened IntelliJ IDEA in the browser at port 3001. The container exposes ports 3000 and 3001 to the host.

```
cd ~
git clone https://github.com/WaqasSaleem97/Intellij-Idea-docker.git
cd Intellij-Idea-docker
bash setup.sh
docker compose ps
```

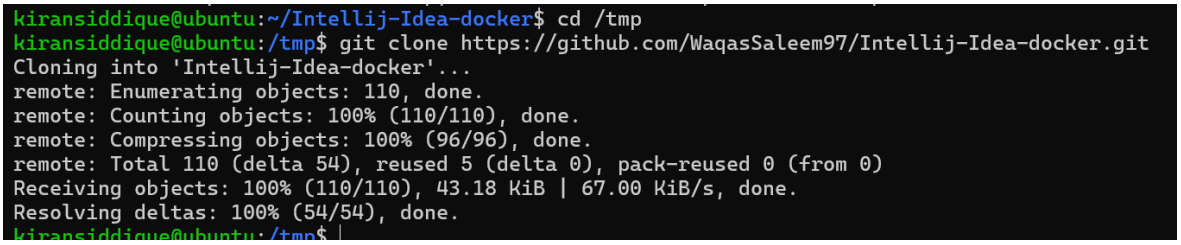


Figure 1: Cloning the IntelliJ Docker repository (intellij_repo_clone.png)

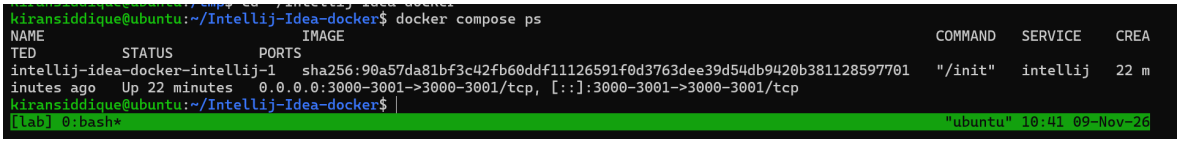


Figure 2: Service intellij running with ports 3000-3001 published (intellij_running.png)

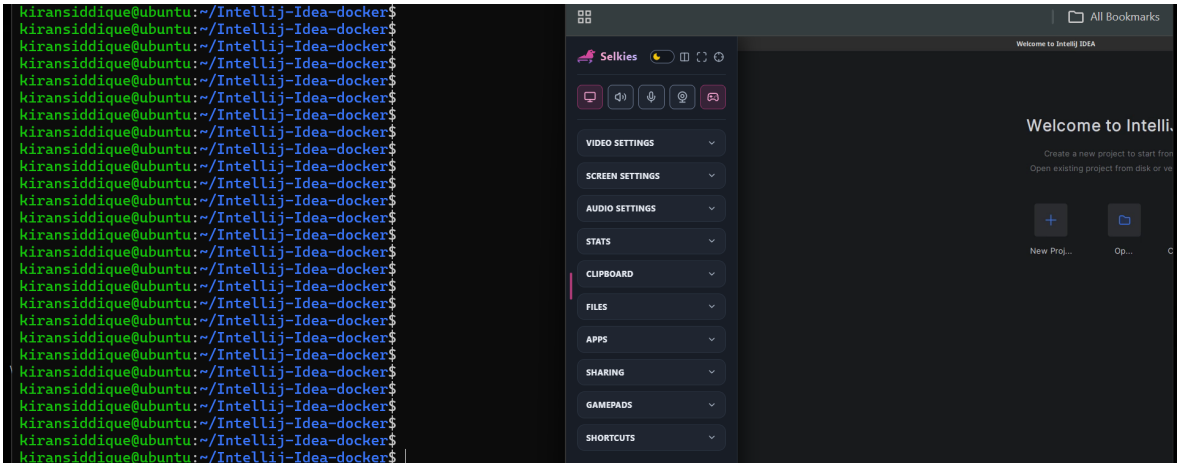


Figure 3: IntelliJ IDEA Welcome screen in the browser desktop, with the Ubuntu terminal beside it (intellij_desktop.png)

Notes on this task. The first build failed because the buildx plugin was missing, so I installed docker-buildx and reran setup.sh. A later build failed with a TLS handshake timeout to ghcr.io; it completed on the next run. The browser desktop also went black several times after the WebSocket disconnected. Restarting the service with docker compose restart intellij restored the display. Long builds were run inside tmux so they survived SSH disconnects.

Task 2: Build and Run the Maven Project

From the Welcome screen I cloned https://gitlab.com/nanuchi/java-maven-app.git into /config/IdeaProjects/java-maven-app. In the IntelliJ terminal I installed Maven, packaged the project, added the Elasticsearch dependency to pom.xml, tested the project, and ran the generated JAR.

```
sudo apt update
sudo apt install -y maven
mvn --version
mvn package
mvn test
java -jar target/java-maven-app-1.1.0-SNAPSHOT.jar
```

Dependency added inside the existing dependencies block of pom.xml:

```
<dependency>
    <groupId>org.elasticsearch</groupId>
    <artifactId>elasticsearch</artifactId>
    <version>7.0.0</version>
    <scope>compile</scope>
</dependency>
```

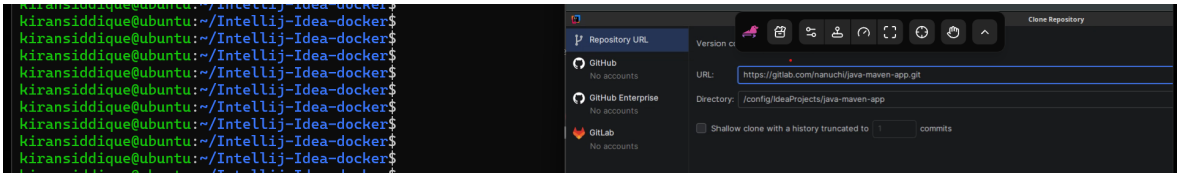


Figure 4: Clone dialog with the GitLab URL and the /config/IdeaProjects/java-maven-app directory (maven_project_clone.png)

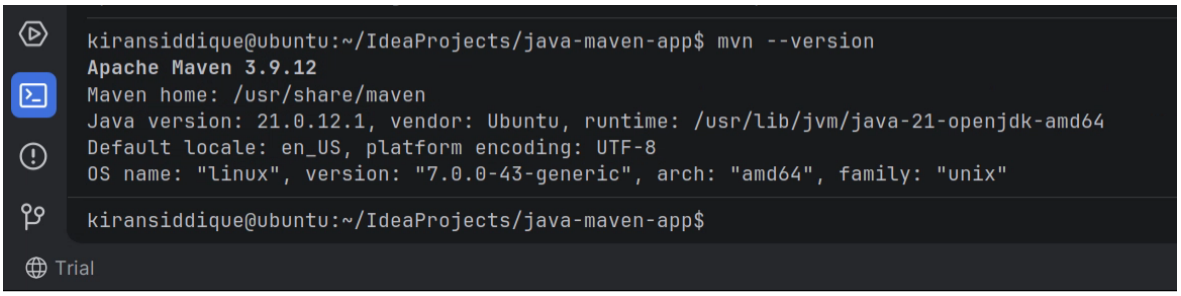


Figure 5: Apache Maven 3.9.12 installed, shown by mvn --version in the IntelliJ terminal (maven_installed.png)

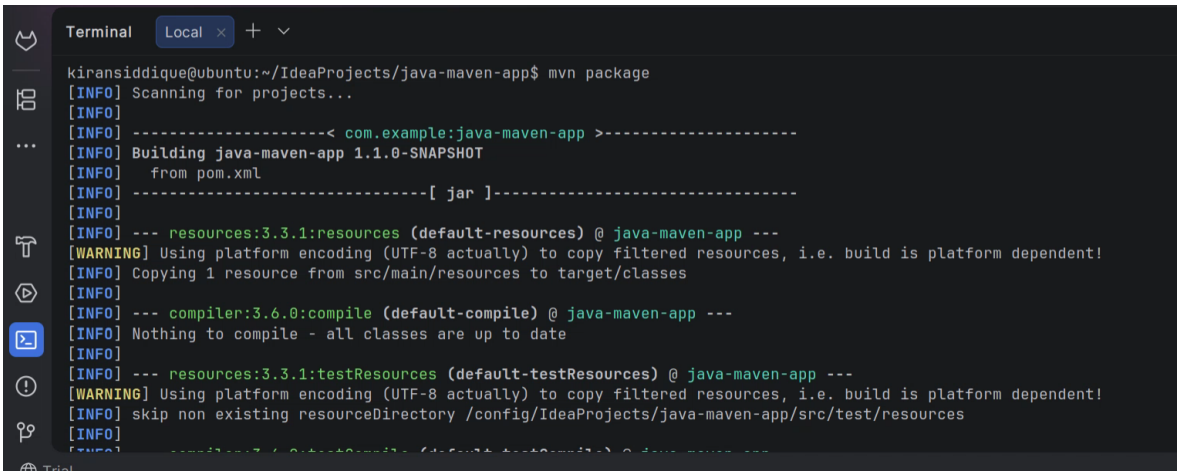


Figure 6: First mvn package finishing with BUILD SUCCESS (maven_package.png)

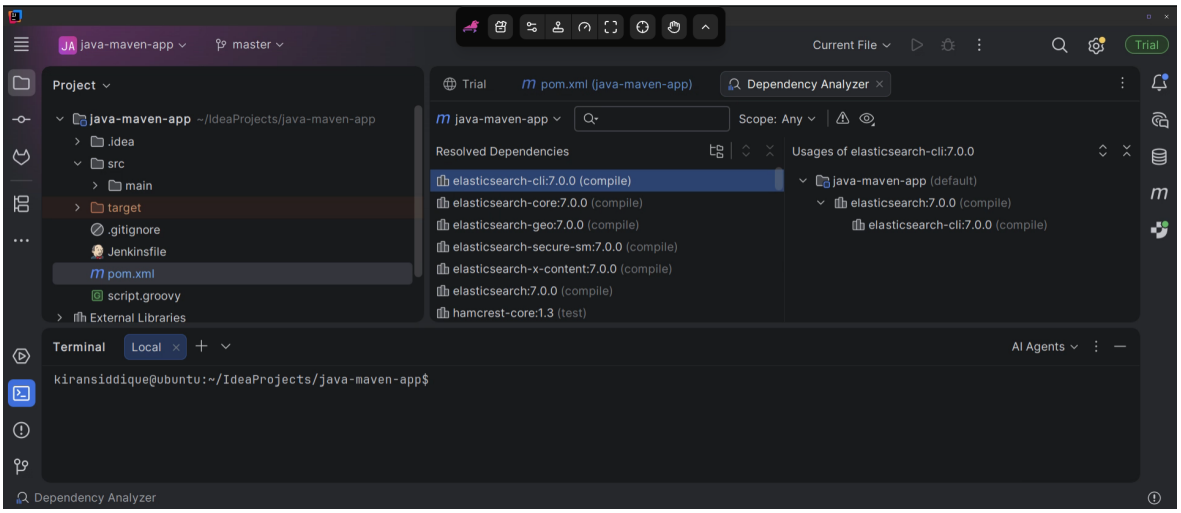


Figure 7: Elasticsearch 7.0.0 resolved after syncing the Maven changes (maven_elasticsearch_dependency.png)

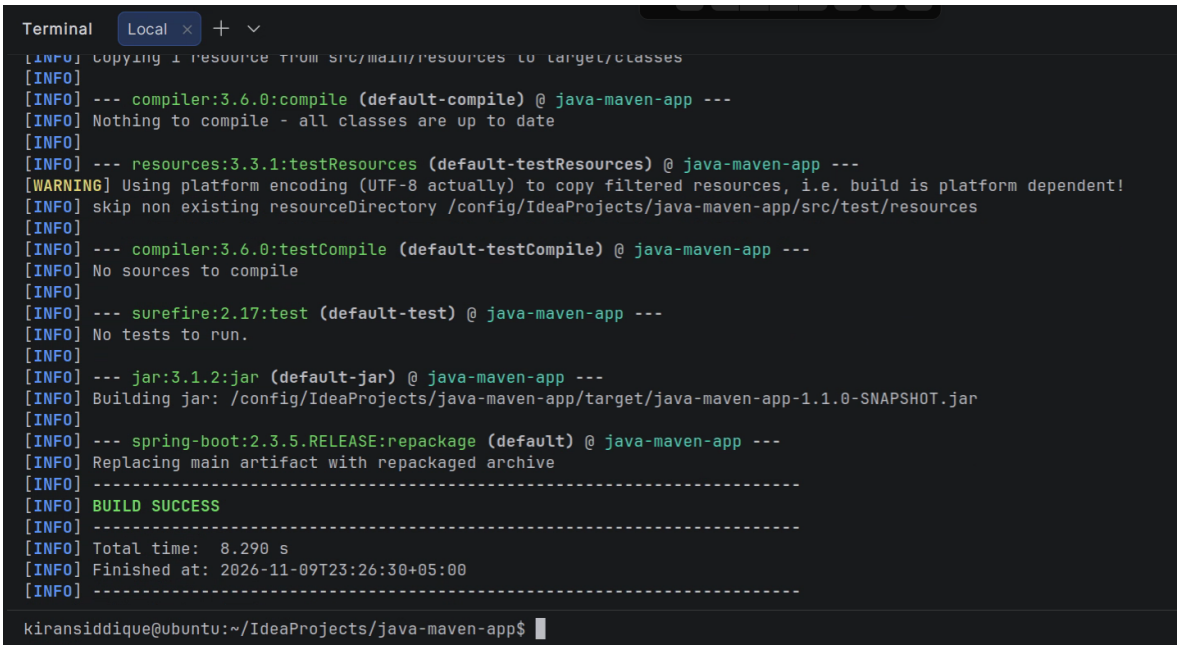


Figure 8: Second mvn package after adding the dependency, BUILD SUCCESS (maven_repackage.png)



Figure 9: Output of mvn test, the test phase of the build (maven_test.png)

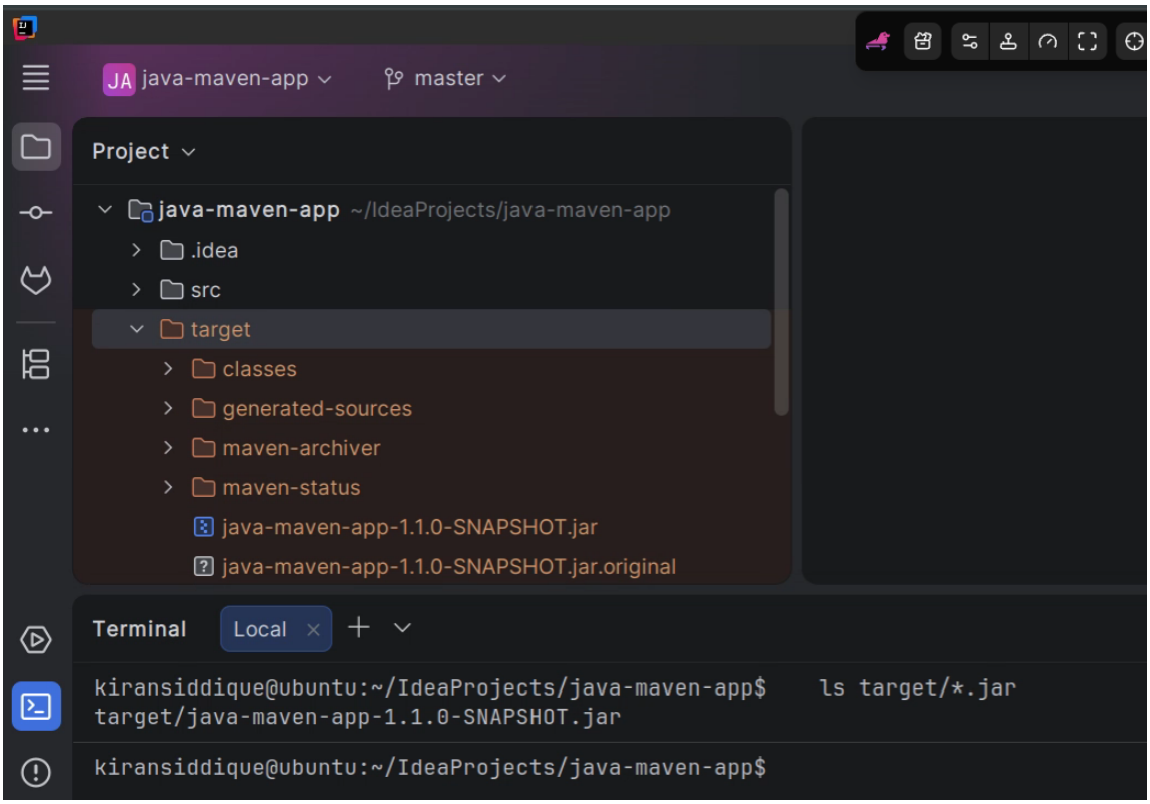


Figure 10: target directory containing java-maven-app-1.1.0-SNAPSHOT.jar and the ls target/.jar output (maven_jar.png)*

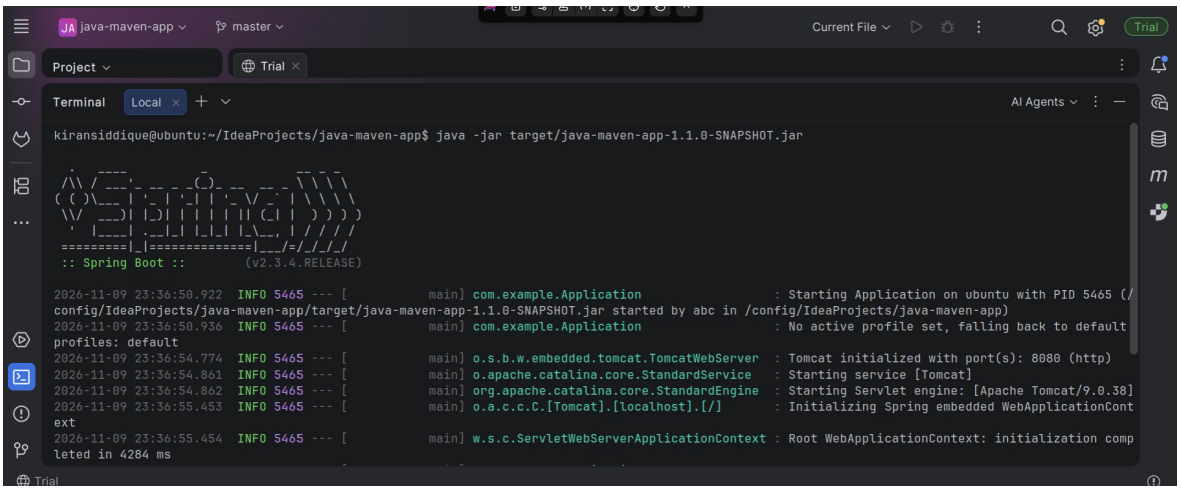


Figure 11: java -jar starting the Spring Boot application on Tomcat, port 8080 (maven_running.png)

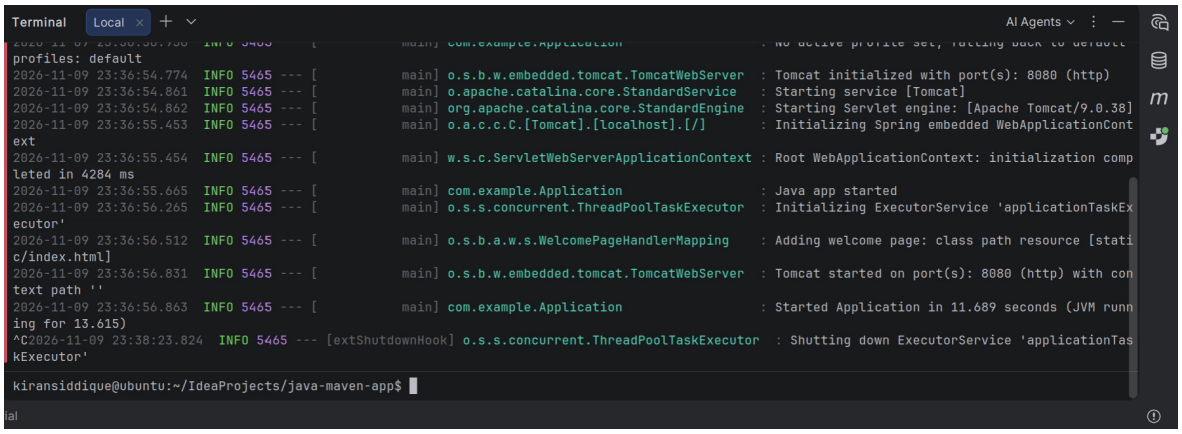


Figure 12: Application stopped with Ctrl+C and the shutdown messages (maven_stopped.png)

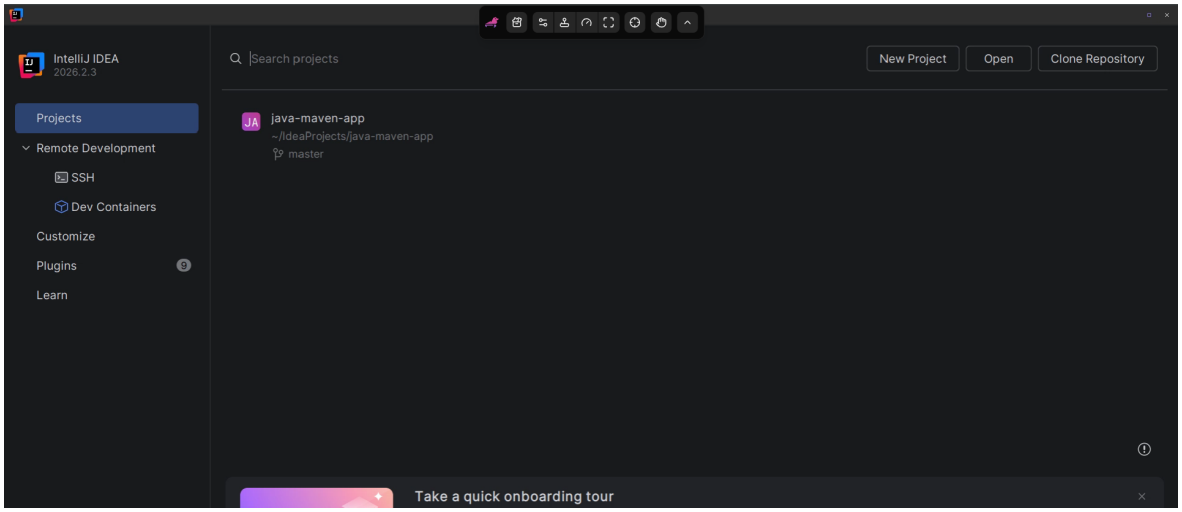


Figure 13: Project closed and IntelliJ back on the Welcome screen (maven_project_closed.png)

Notes on this task. The first mvn package failed with a PluginResolutionException while downloading the maven-compiler-plugin descriptors. curl confirmed that repo.maven.apache.org was reachable, and mvn -U package then completed with BUILD SUCCESS and downloaded the missing artifacts.

Task 3: Build and Run the Gradle Project

I cloned https://gitlab.com/nanuchi/java-app.git into /config/IdeaProjects/java-app. The project's Gradle 7.0 wrapper cannot run on Java 21, and the system Java was 21. I therefore set the Project SDK to JDK 16.0.2 (/config/.jdks/openjdk-16.0.2), set the language level to 8, and set the Gradle JVM to the project SDK. Then I verified the JVM from the terminal.

```
cd /config/IdeaProjects/java-app
chmod +x gradlew
./gradlew --version
./gradlew build
```

Dependency added inside the dependencies block of build.gradle:

```
implementation("org.elasticsearch:elasticsearch:7.0.0")
```

```
./gradlew build
ls build/libs/
java -jar build/libs/java-app-1.0-SNAPSHOT.jar
```

Notes on this task. Before the JDK change, ./gradlew --version reported JVM 21.0.12.1. After the change, with JAVA_HOME set to /config/.jdks/openjdk-16.0.2, the same command reported JVM 16.0.2 (Oracle Corporation 16.0.2+7-67).

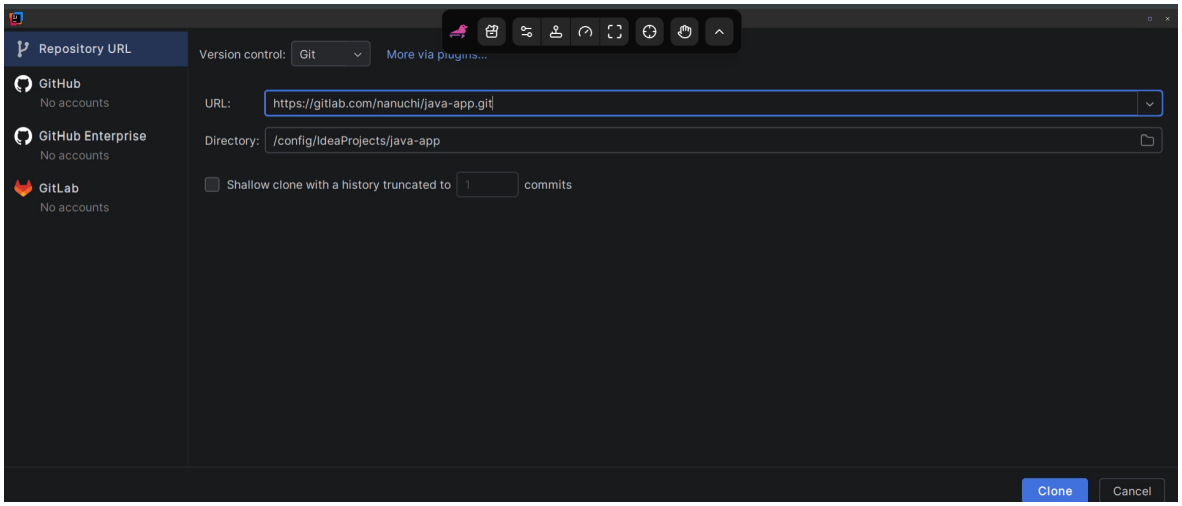


Figure 14: Clone dialog for https://gitlab.com/nanuchi/java-app.git (gradle_project_clone.png)



Figure 15: First ./gradlew build finishing with BUILD SUCCESSFUL on JDK 16 (gradle_build.png)

[Screenshot pending: gradle_elasticsearch_dependency.png]

Figure 16: build.gradle with the Elasticsearch 7.0.0 dependency and the synced External Libraries (gradle_elasticsearch_dependency.png)

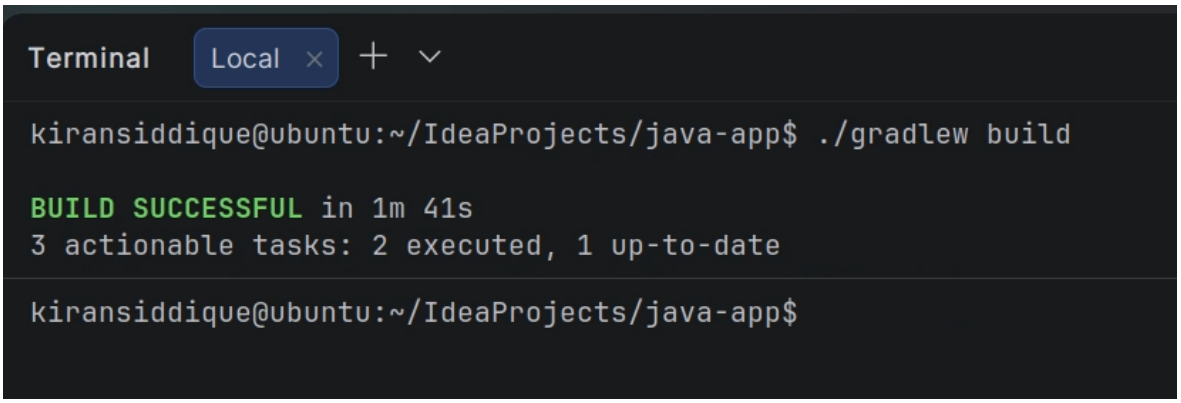


Figure 17: Rebuild after the dependency change, BUILD SUCCESSFUL (gradle_rebuild.png)

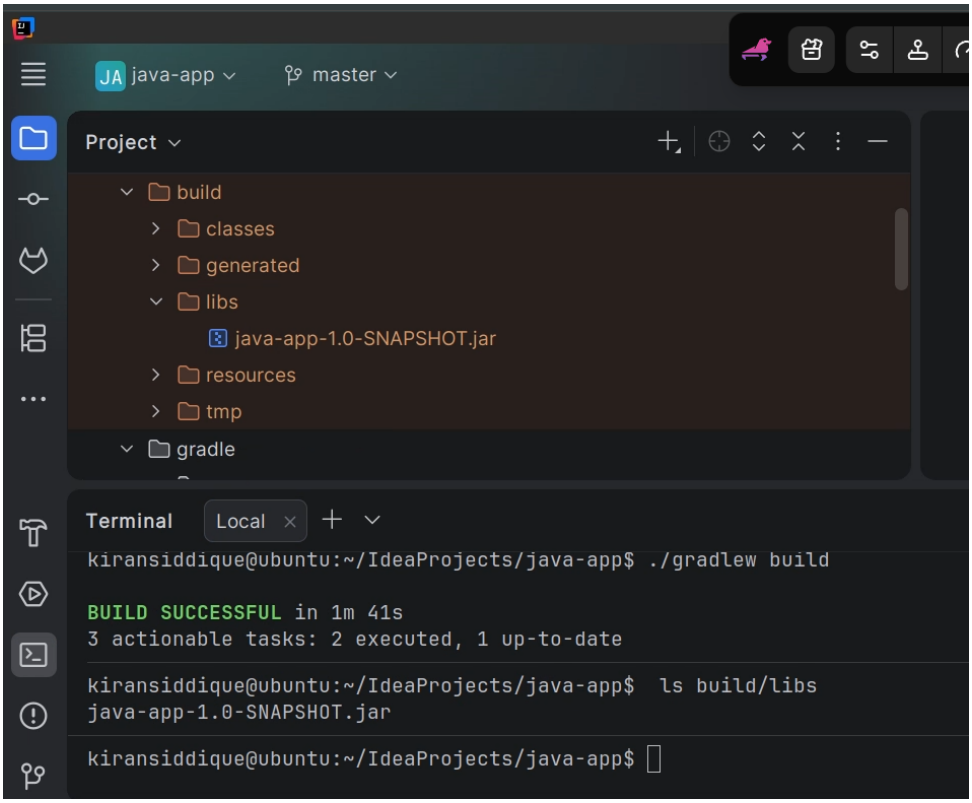


Figure 18: build/libs directory containing java-app-1.0-SNAPSHOT.jar (gradle_jar.png)

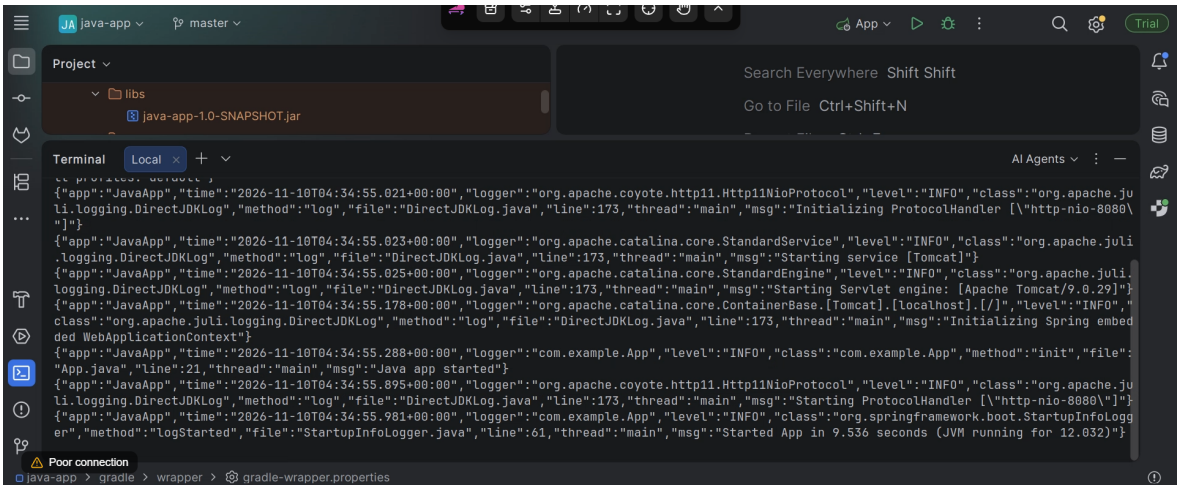


Figure 19: Application started with java -jar build/libs/java-app-1.0-SNAPSHOT.jar (gradle_running.png)

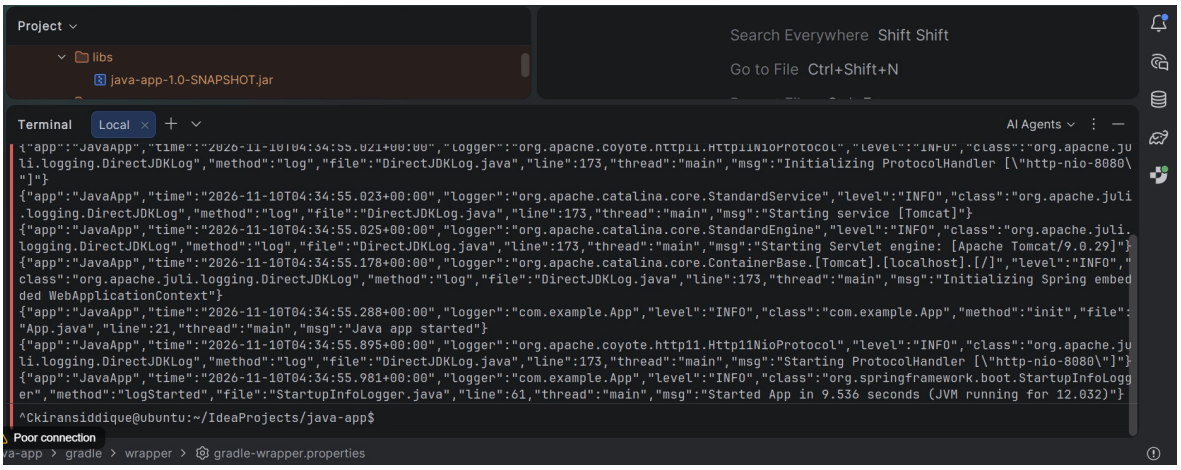


Figure 20: Application stopped with Ctrl+C and the prompt returned (gradle_stopped.png)

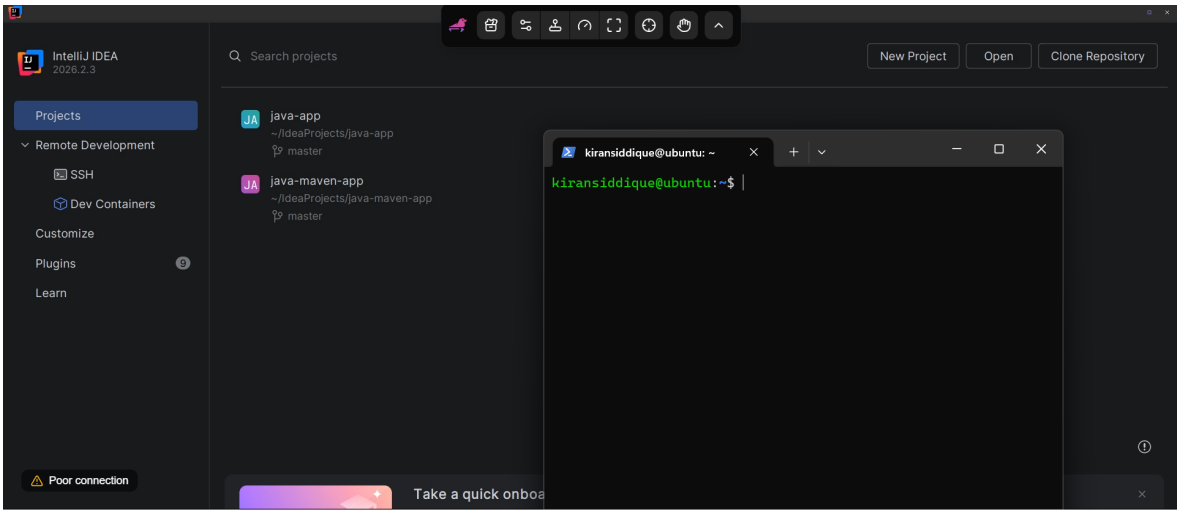


Figure 21: Project closed and IntelliJ back on the Welcome screen (gradle_project_closed.png)

Task 4: Install and Run the Node.js Project

I cloned https://gitlab.com/devopsbootcamp3/node-project.git into /config/IdeaProjects/node-project and installed Node.js and npm. The application runs from the app directory. The supplied server listens on port 3000, which conflicts with the desktop container, so I changed the listener to port 4000.

```
sudo apt update
sudo apt install -y nodejs npm
node --version
npm --version
cd app
npm install
npm start
npm test
npm pack
```

Change in app/server.js:

```
app.listen(4000, function () {
  console.log("app listening on port 4000!");
});
```

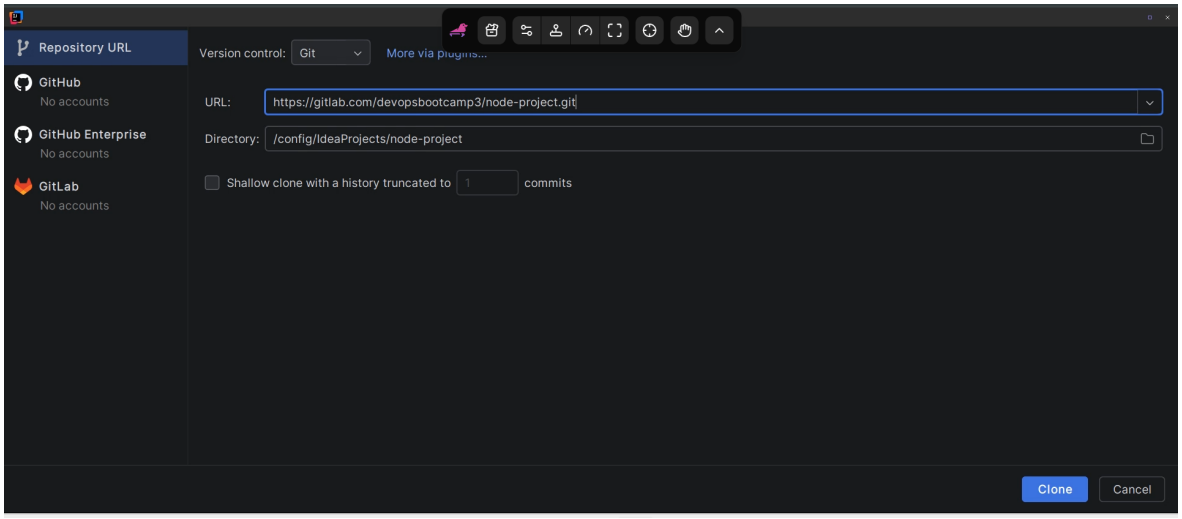


Figure 22: Clone dialog for https://gitlab.com/devopsbootcamp3/node-project.git (node_project_clone.png)



Figure 23: Node.js and npm versions after installation (nodejs_installed.png)



Figure 24: npm install completed in the app directory (npm_install.png)



Figure 25: npm start with the message app listening on port 4000! (node_running.png)

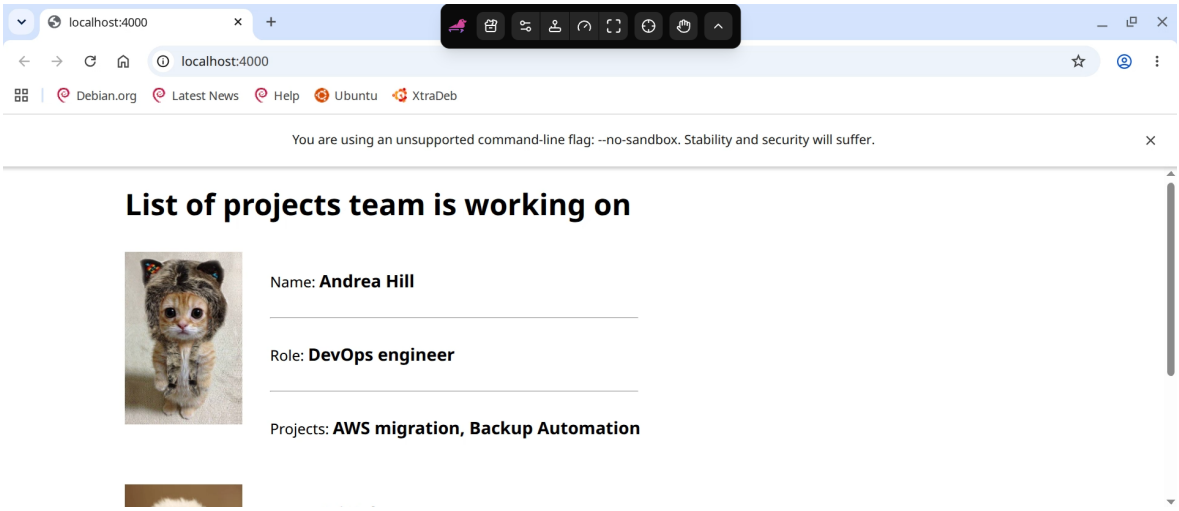


Figure 26: Node.js project page at http://localhost:4000 in the container browser, with the IntelliJ terminal (node_browser.png)

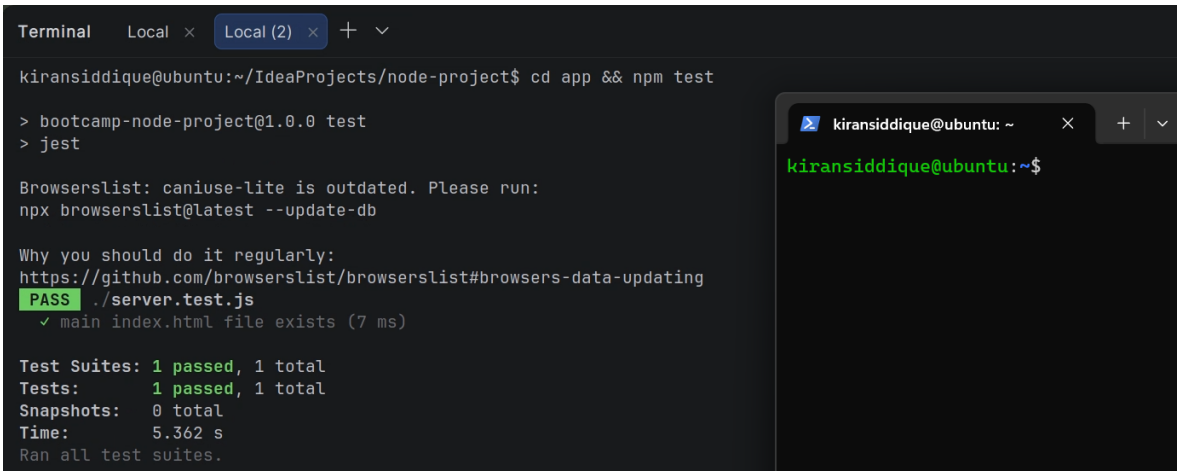


Figure 27: npm test (Jest) with the passing test summary (npm_test.png)

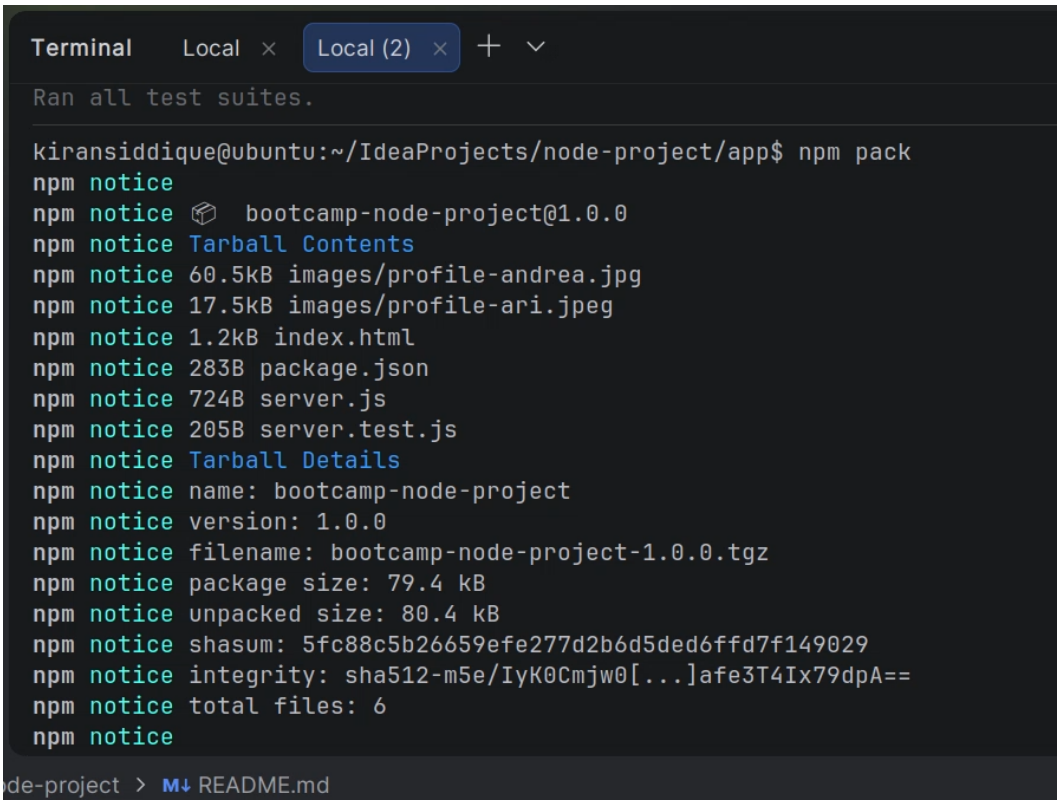


Figure 28: npm pack creating bootcamp-node-project-1.0.0.tgz (npm_pack.png)

Task 5: Build the React and Node.js API Artifact

I cloned https://github.com/bbachi/react-nodejs-example.git into /config/IdeaProjects/react-nodejs-example, installed the API dependencies in the api directory, built the bundle with webpack, and inspected the generated file.

```
cd api
npm install
npm run build
cat server.bundle.js | more
```

If the build reports ERR_OSSL_EVP_UNSUPPORTED on this Node.js version, the same build runs with NODE_OPTIONS=--openssl-legacy-provider npm run build.

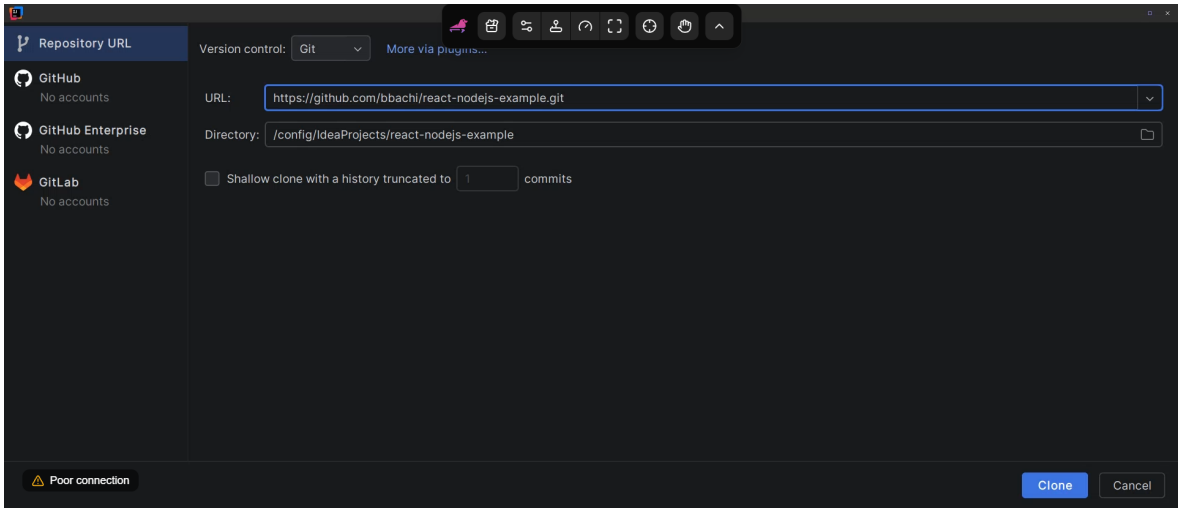


Figure 29: Clone dialog for https://github.com/bbachi/react-nodejs-example.git (react_node_project_clone.png)

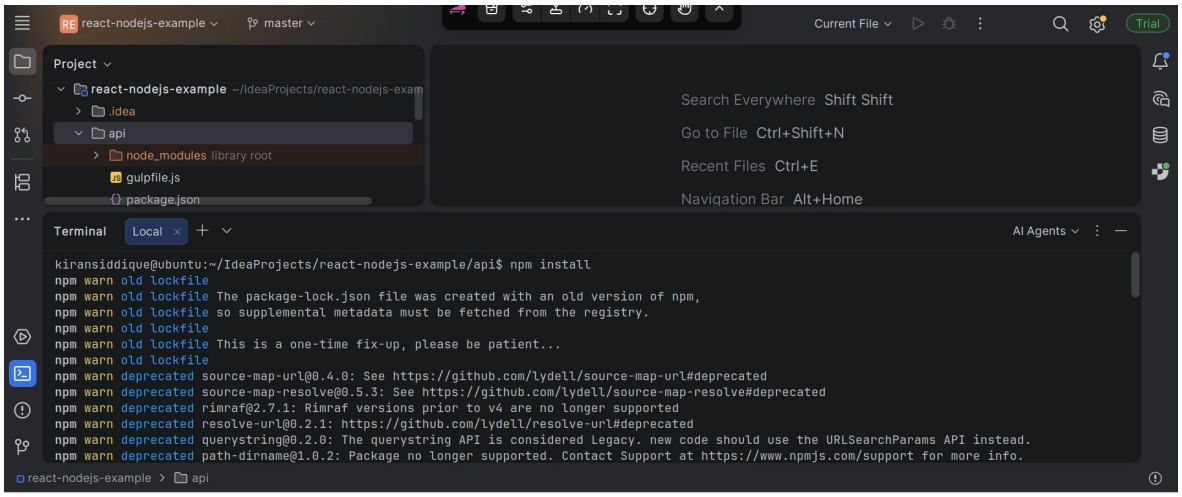


Figure 30: Completed npm install and the api/node_modules directory (react_api_dependencies.png)

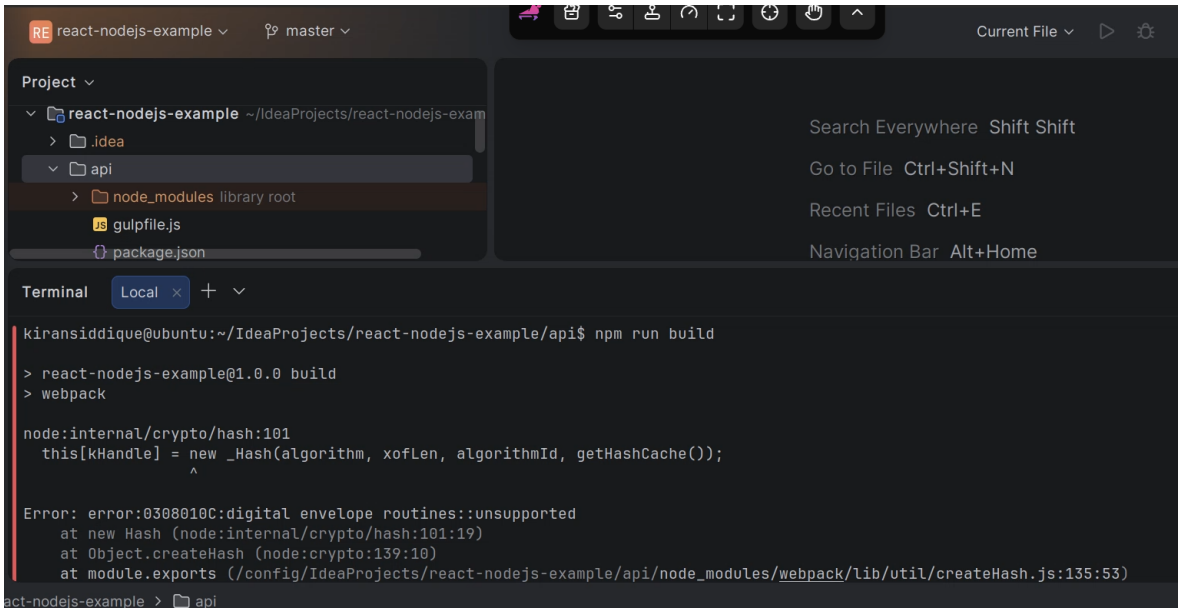


Figure 31: npm run build (webpack) producing api/server.bundle.js (react_api_build.png)

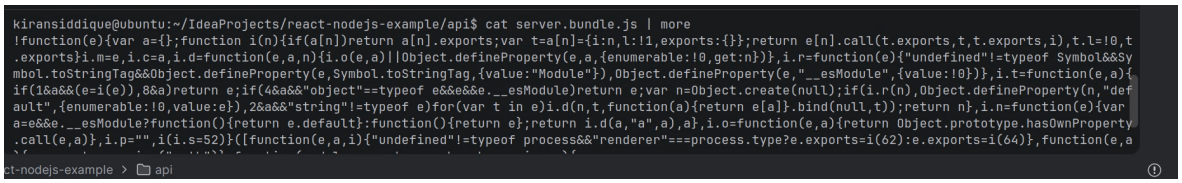


Figure 32: Bundled JavaScript viewed with more, then exited with q (react_api_bundle.png)

Task 6: Build a Docker Image for the Node.js Project

From the node-project root, where the Dockerfile sits beside the app folder, I checked the Docker installation, listed the images, built the image lab5:1.0, and listed the images again to confirm the repository and tag.

```
docker --version
ls
sudo docker images
sudo docker build -t lab5:1.0 .
sudo docker images
```

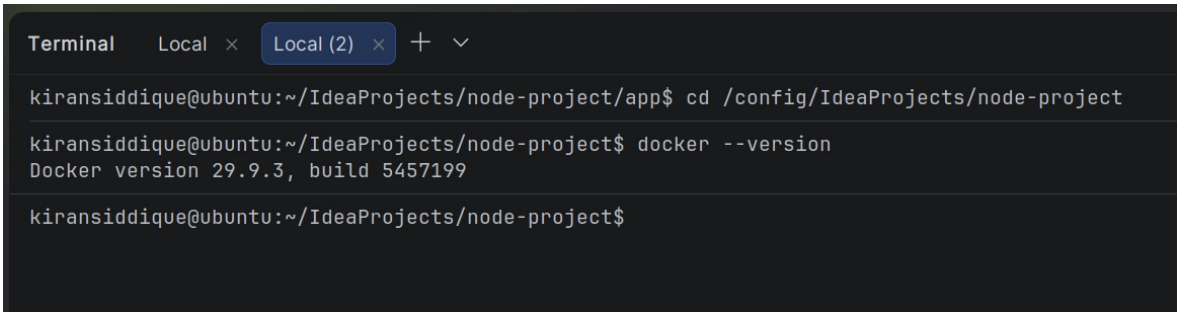


Figure 33: docker --version output (docker_installed.png)



Figure 34: Dockerfile listed in the repository root (dockerfile_exists.png)

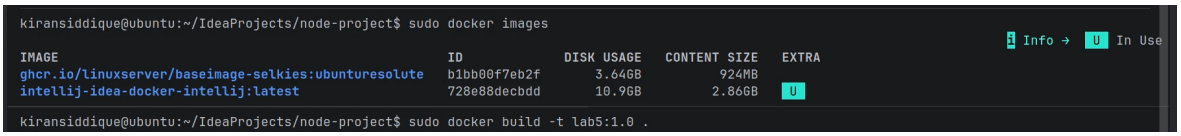


Figure 35: sudo docker images before the build (docker_images_before.png)

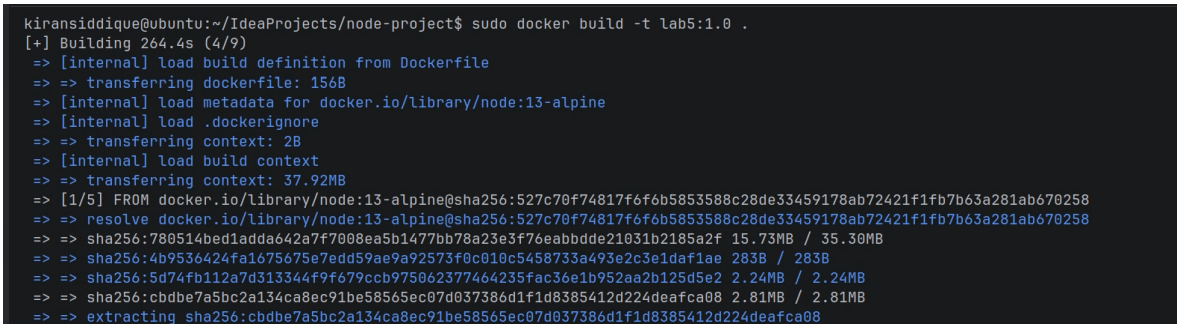


Figure 36: sudo docker build -t lab5:1.0 . completing successfully (docker_build.png)

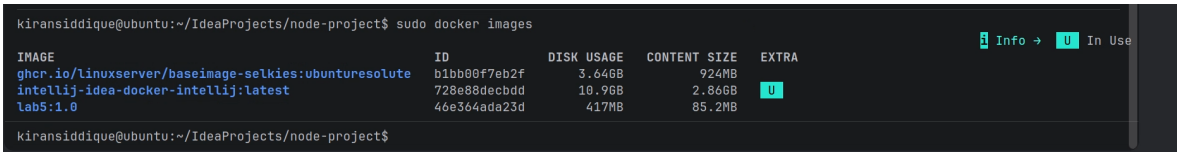


Figure 37: sudo docker images showing repository lab5, tag 1.0 and its image ID (docker_images_after.png)

Exam Evaluation Questions

Question 1: Maven built java-maven-app-1.1.0-SNAPSHOT.jar. Which values in pom.xml determine this name, and what does the package phase do?

Answer: The file name is built as artifactId-version.jar. The artifactId java-maven-app and the version 1.1.0-SNAPSHOT come from the project coordinates at the top of pom.xml. The package phase runs after validate, compile and test. It compiles the sources into target/classes, runs the tests, and then creates the JAR in target. The spring-boot-maven-plugin repackage goal then rewrites that JAR as an executable archive with its dependencies inside it, which is why java -jar works. Maven keeps the pre-repackage copy as java-maven-app-1.1.0-SNAPSHOT.jar.original.

Question 2: Why can ./gradlew build work without a separate system-wide Gradle installation? Which JDK settings would you check if this sample reports an unsupported Java version?

Answer: The wrapper reads gradle/wrapper/gradle-wrapper.properties, which names the exact Gradle version. On the first run the gradlew script downloads that distribution into the Gradle user home and uses it, so no system Gradle is needed. If the sample reports an unsupported Java version, check the Project SDK and language level in File > Project Structure, the Gradle JVM in File > Settings > Build Tools > Gradle, and the JAVA_HOME and PATH used by the terminal. ./gradlew --version shows which JVM the wrapper actually runs. In this lab the sample ran on Java 21 and failed. After the project and Gradle JVM were set to JDK 16.0.2, the same command reported JVM 16.0.2.

Question 3: Why does EADDRINUSE occur on port 3000 in this environment, and how does changing the application listener to port 4000 let the container's browser open it?

Answer: EADDRINUSE means another process already listens on that address and port. Inside this desktop container, port 3000 is already in use by the container's own services, so Node.js cannot bind to it. Changing the listener to 4000 gives the application a free port. The application and the browser in the container share the same network namespace, so the browser can reach the app at http://localhost:4000 even though 4000 is not published to the host.

Summary

In this lab I launched the IntelliJ IDEA Docker environment and used IntelliJ to clone four projects. For the Maven project I added Elasticsearch 7.0.0 to pom.xml, ran the package and test phases, and ran the generated JAR. For the Gradle project I set the Project SDK and Gradle JVM to JDK 16.0.2, added the same dependency to build.gradle, rebuilt the project, and ran the generated JAR. For the Node.js project I installed dependencies, moved the server to port 4000, ran the Jest tests and packaged the application with npm pack. I built the React and Node.js API bundle with webpack and inspected it, and finally I built the Node.js project into the Docker image lab5:1.0. Along the way I resolved several environment problems, including the missing buildx plugin, a Maven plugin resolution failure, and the Java 21 versus Gradle 7.0 mismatch.